API & Integration Guide

How the components talk: REST, SOAP, the mashup and the gateway

For the professor and the team. It explains the request path, authentication, every endpoint, the error format, the legacy SOAP integration and the mashup, follows one complete scenario through every component, and ends with the questions we expect at the defense. The examples are real exchanges captured from the running system.

Live system	https://opendoor-rust.vercel.app
API gateway	`https://gateway-production-4e58.up.railway.app/api`
Contract	`docs/openapi.yaml` (OpenAPI 3.1) and the WSDL at `/soap?wsdl`
Examples	Captured October 24, 2026 from the local stack
Version	October 24, 2026 · commit `75c8dd5`

OpenDoor – PWD job matching by workplace accommodations · SIA 1 and IPT final project · Group 3 – ICARUS, IT31S1

Contents

1 About this guide

2 Architecture and request path

3 API basics: format, authentication, CORS, limits

4 Endpoint reference (73 operations)

5 Errors: envelope, codes and real examples

6 Legacy integration: the SOAP Business Registry

7 The mashup: Workplace Accessibility Snapshot

8 End-to-end scenario through every component

- 8.1 Employer registers and fills in the company profile
- 8.2 Admin checks the employer with the SOAP registry and approves
- 8.3 The verified employer publishes a job
- 8.4 Candidate finds the job by its match score and opens the snapshot
- 8.5 Candidate applies; employer reviews
- 8.6 The hire is reported by one side and confirmed by the other
- 8.7 From day 30 after the start date: feedback builds the trust score

9 Security summary

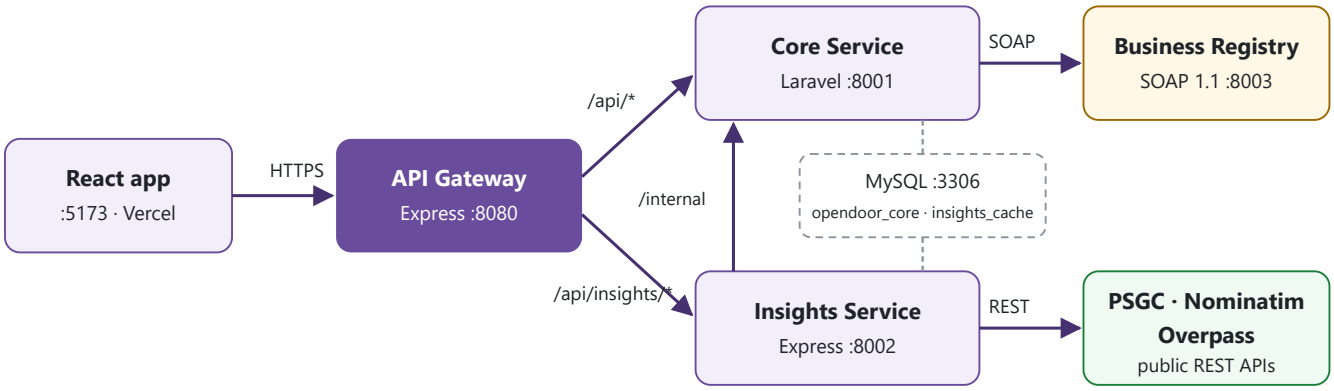
10 Questions we expect, with answers

1 About this guide

How OpenDoor's parts talk to each other and to the outside world, for anyone who has to explain or extend the integration.

- **Companion:** the *Run & Demo Guide* shows how to start the system and follows nine clicks from the screen to the database. This guide is the reference behind it.
- **Examples are real:** every request and response in sections 3–7 was sent to the running system (local stack, freshly seeded database). Long lists are shortened to 2 items.
- **Secrets** (passwords, tokens, the service key, the SOAP credentials) are shown as `•••••• (hidden)`.

2 Architecture and request path



Link	Style	Security
Browser → Gateway	REST/JSON over HTTPS	CORS (app origin only), rate limit, bearer token passed through
Gateway → Core, Insights	HTTP proxy, path unchanged, `X-Request-ID` and `X-Forwarded-For` added	Private network; the services are not public
Insights → Core	REST/JSON `GET /internal/job-postings/{id}/summary`	`X-Service-Key` (constant-time compare)
Core → Business Registry	SOAP 1.1 / XML, WSDL, document/literal	WS-Security UsernameToken
Insights → PSGC, Nominatim, Overpass	REST/JSON over HTTPS (Overpass QL query)	Public APIs; User-Agent with contact email for Nominatim
Core, Insights → MySQL	SQL (Eloquent / mysql2)	One database user per service: `opendoor_core`, `insights_cache`

Gateway routing

Incoming path	Goes to	Timeout
`/api/health`	Gateway itself (pings Core and Insights)	3 s per service
`/api/insights/*`	Insights Service	15 s
`/api/*` (everything else)	Core Service	10 s
`/internal/*`	never routed – 404	–

The gateway does not read bodies or check tokens – it forwards them so uploads stream through and Core stays the single place that decides who may do what. When a service is down or slow, the gateway itself answers with the error envelope (503 / 502 / 504, section 5).

3 API basics: format, authentication, CORS, limits

Base URL	Local `http://localhost:8080/api` · production `https://gateway-production-4e58.up.railway.app/api`. The React app knows only this address (`VITE_API_URL`).
Format	REST over HTTPS, JSON request and response bodies (`Content-Type: application/json`, `Accept: application/json`). Résumé and logo uploads use `multipart/form-data`.
Authentication	`POST /api/login` returns a Laravel Sanctum token. Send it as `Authorization: Bearer <token>`. Core checks it on every protected route; the gateway passes the header through unchanged.
Roles	**Guest** (no token), **Candidate**, **Employer**, **Admin**; service-to-service calls use a service key instead. A wrong role gets 403, no or a revoked token 401.
Request ID	The gateway gives every request an `X-Request-ID` (UUID v4, or keeps the caller's) and passes it to the services. It comes back in the response header and in every error body as `request_id`.
Lists	12 items per page, `?page=`; the response has `data` and `meta` (current page, last page, total).
Dates	ISO 8601 with the Manila offset (`2026-10-19T23:29:24+08:00`); dates without time as `YYYY-MM-DD`.
CORS	Only the app's origin (`CORS_ALLOWED_ORIGIN`: the Vercel domain in production, `http://localhost:5173` locally) gets `Access-Control-Allow-Origin`. The gateway drops any CORS headers the services send, so it alone decides.
Rate limits	Gateway: 120 requests per minute per IP in production (`RATE_LIMIT_PER_MIN`; 300 on the local machine for testing) → 429 `RATE_LIMITED` with `Retry-After`. Core additionally allows 5 log-in (and 5 password-reset) attempts per minute per email + IP.
Contract	The full machine-readable contract is `docs/openapi.yaml` (OpenAPI 3.1). Section 4 is generated from it.

Log in and use the token

REQUEST

```
POST http://127.0.0.1:8080/api/login
Content-Type: application/json

{
  "email": "candidate@opendoor.test",
  "password": "•••••• (hidden)"
}
```

RESPONSE

```
HTTP 200
content-type: application/json
x-request-id: a53e05e4-b649-4016-929c-8d4fd49797f5
ratelimit-policy: 300;w=60
vary: Origin

{
  "token": "5|•••••• (hidden)",
  "user": {
    "id": 2,
    "name": "Maria Santos",
    "email": "candidate@opendoor.test",
    "role": "candidate",
    "is_active": true,
    "display_prefs": null,
    "email_notifications": true,
    "onboarding": {
      "tour_done": true,
      "checklist_hidden": true
    },
    "profile_complete": true
  }
}
```

REQUEST

```
GET http://127.0.0.1:8080/api/me
Authorization: Bearer 6|•••••• (hidden)
```

RESPONSE

```
HTTP 200
content-type: application/json
x-request-id: eacc9a76-ebed-4dd9-a3c2-e4d1c56f5117
ratelimit-policy: 300;w=60
vary: Origin

{
```

```
  "id": 2,
  "name": "Maria Santos",
  "email": "candidate@opendoor.test",
  "role": "candidate",
  "is_active": true,
  "display_prefs": null,
  "email_notifications": true,
  "onboarding": {
    "tour_done": true,
    "checklist_hidden": true
  },
  "profile_complete": true
}
```

Gateway health

REQUEST

```
GET http://127.0.0.1:8080/api/health
```

RESPONSE

```
HTTP 200
content-type: application/json; charset=utf-8
x-request-id: 43071382-019d-42ff-9c85-c5965b832d9c
ratelimit-policy: 300;w=60
vary: Origin

{
  "status": "ok",
  "services": {
    "core": "up",
    "insights": "up"
  }
}
```

CORS preflight

From the app's origin the gateway allows the call; from any other site it answers without `Access-Control-Allow-Origin`, so the browser blocks the real request.

REQUEST

```
OPTIONS http://127.0.0.1:8080/api/login
Origin: http://localhost:5173
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type
```

RESPONSE

```
HTTP 204
x-request-id: 8f81eee9-6ceb-4466-91a2-8a1fc5ee4f22
access-control-allow-origin: http://localhost:5173
access-control-allow-methods: GET,HEAD,PUT,PATCH,POST,DELETE
access-control-allow-headers: content-type
vary: Origin, Access-Control-Request-Headers
```

REQUEST

```
OPTIONS http://127.0.0.1:8080/api/login
Origin: https://example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type
```

RESPONSE

```
HTTP 204
x-request-id: 2aa3d639-df96-45e6-a82b-0225b571aa97
access-control-allow-methods: GET,HEAD,PUT,PATCH,POST,DELETE
access-control-allow-headers: content-type
vary: Origin, Access-Control-Request-Headers
```

4 Endpoint reference

Generated from `docs/openapi.yaml`: 73 operations in 16 groups (M1–M13 are the SIA modules).

- **Who:** Public = no token; Signed in = any role; Candidate / Employer / Admin = that role only (403 otherwise); owner = only for their own data (Laravel Policy).
- **Request / Success:** JSON fields (`*` = required) or a schema name from `openapi.yaml` (`components/schemas`). Lists are `{data, meta}`, 12 per page.
- **Errors:** the HTTP statuses the operation documents; every one uses the envelope of section 5. 401 applies to every non-public operation and 429 to all.

Gateway

Method and path	Who	What	Request	Success	Errors
GET `/api/health`	Public	Gateway health with downstream service status	–	**200** {status*, services*}	–

M1 Auth

Method and path	Who	What	Request	Success	Errors
POST `/api/register`	Public	Register a candidate or employer (admins never self-register)	{name*, email*, password*, password_confirmation*, role*, consent*}	**201** `AuthResponse`	422
POST `/api/login`	Public	Log in and receive a Sanctum token (5 attempts per minute)	{email*, password*}	**200** `AuthResponse`	403, 422, 429
POST `/api/forgot-password`	Public	Email a link to reset the password (5 requests per minute)	{email*}	**200** {message*}	422, 429
POST `/api/reset-password`	Public	Set a new password with the emailed link (5 requests per minute)	{token*, email*, password*, password_confirmation*}	**200** {message*}	422, 429
POST `/api/logout`	Signed in	Revoke the current token	–	**204** no body	401
GET `/api/me`	Signed in	Current user and role	–	**200** `Account`	401
PUT `/api/me/password`	Signed in	Change password	{current_password*, password*, password_confirmation*}	**204** no body	401, 422
PATCH `/api/me/onboarding`	Candidate, Employer	Remember the welcome tour and the "Get started" list	{tour_done, checklist_hidden}	**200** `Onboarding`	401, 403, 422

M2 Candidate profile

Method and path	Who	What	Request	Success	Errors
GET `/api/candidate/profile`	Candidate	Own candidate profile including accommodation needs	–	**200** `CandidateProfile`	401, 403
PUT `/api/candidate/profile`	Candidate	Update own candidate profile	{headline, skills, experience, education_level, experience_years, preferred_work_setups, preferred_employment_types, location_psgc, location_label, +2}	**200** `CandidateProfile`	422
PUT `/api/candidate/accommodations`	Candidate	Replace the candidate's accommodation needs	{accommodation_ids*}	**200** {data}	422

Method and path	Who	What	Request	Success	Errors
POST `/api/candidate/resume`	Candidate	Upload résumé (PDF only, max 5 MB)	{resume*} (multipart/form-data)	**200** {resume_url}	422
GET `/api/candidate/resume/skill-suggestions`	Candidate	Skills from the skills list found in the uploaded résumé (Decision 2026-10-21)	–	**200** {data}	401, 403, 404
GET `/api/resumes/{id}`	Public	Open a résumé PDF from a temporary signed link	–	**200** string	403, 404

M3 Company

Method and path	Who	What	Request	Success	Errors
GET `/api/employer/company`	Employer	Own company profile	–	**200** `Company`	403, 404
PUT `/api/employer/company`	Employer	Create or update own company profile (changing business_reg_no clears verification)	{company_name*, business_reg_no*, registry_type*, industry*, address*, location_psgc, location_label, description, logo} (multipart/form-data)	**200** `Company`	422
GET `/api/employers/{id}`	Public	Public employer page (company, verified badge, open postings, trust score)	–	**200** `EmployerPublic` + {open_postings}	404
GET `/api/employers/{id}/logo`	Public	Company logo image (served through the gateway; the client never calls Core directly)	–	**200** string	404
GET `/api/admin/employers`	Admin	Employer verification queue (admin)	–	**200** {data, meta}	403
GET `/api/admin/employers/{id}`	Admin	One employer with owner and latest registry check (admin)	–	**200** `AdminEmployer`	403, 404
PATCH `/api/admin/employers/{id}/verify`	Admin	Run a SOAP registry check, or approve / reject an employer	{action*, reason}	**200** {registry, verified_at}	403, 404, 422, 502

M4 Job postings

Method and path	Who	What	Request	Success	Errors
GET `/api/employer/job-postings`	Employer	Own postings (all statuses) with applicant counts	–	**200** {data, meta}	–
POST `/api/job-postings`	Employer	Create a posting (draft, or publish)	{title*, description*, responsibilities, skills, education_level, experience_years, qualifications, salary_min, salary_max, +14}	**201** `JobPosting`	403, 422
PUT `/api/job-postings/{id}`	Employer (owner)	Update own posting	{title*, description*, responsibilities, skills, education_level, experience_years, qualifications, salary_min, salary_max, +14}	**200** `JobPosting`	403, 422
DELETE `/api/job-postings/{id}`	Employer (owner)	Soft-delete own posting	–	**204** no body	403

Method and path	Who	What	Request	Success	Errors
PATCH `/api/job-postings/{id}/status`	Employer (owner)	Open or close own posting	{status*}	**200** `JobPosting`	403, 422
GET `/api/admin/job-postings`	Admin	Every posting, or one status (admin)	–	**200** {data, meta}	403
DELETE `/api/admin/job-postings/{id}`	Admin	Remove a posting (admin)	{reason*}	**204** no body	401, 403, 404, 422

M5 Search and matching

Method and path	Who	What	Request	Success	Errors
GET `/api/job-postings`	Public (candidate: match)	Search open postings; "Recommended for me" ranking for candidates	–	**200** {data, meta}	422
GET `/api/job-postings/{id}`	Public (candidate: match)	Posting details (+ `match` for candidates)	–	**200** `JobPosting`	404

M6 Applications

Method and path	Who	What	Request	Success	Errors
POST `/api/job-postings/{id}/applications`	Candidate	Apply to an open posting	{cover_note, interview_request}	**201** `Application`	403, 404, 409, 422
GET `/api/applications`	Candidate (own), Employer (own postings)	Own applications (candidate) or applications to own postings (employer)	–	**200** {data, meta}	403, 422
GET `/api/applications/{id}`	Candidate (own), Employer (own postings)	Application details (employer opening a pending application marks it reviewed)	–	**200** `Application`	403, 404
PATCH `/api/applications/{id}/status`	Candidate, Employer	Change application status (see the status workflow)	{status*}	**200** `Application`	403, 422
POST `/api/applications/{id}/hiring-reports`	Candidate, Employer (of this application)	Report a hire (employer "Report hiring", candidate "Report that I was hired")	{start_date*}	**201** `Application`	403, 404, 409, 422
POST `/api/applications/{id}/hiring-reports/{reportId}/response`	The other party of the report	Answer the other party's hiring report (Confirm, Not yet, Incorrect)	{answer*, start_date}	**200** `Application`	403, 404, 409, 422

M7 Feedback and trust score

Method and path	Who	What	Request	Success	Errors
GET `/api/applications/{id}/feedback-form`	Candidate (hired)	Accommodations to rate (those advertised when the hire was confirmed)	–	**200** {opens_on, is_open, submitted, items}	401, 403, 422
POST `/api/applications/{id}/feedback`	Candidate (hired, day 30+ after the start date)	Submit feedback once, from calendar day 30 after the confirmed start date (Asia/Manila)	{items*, comments}	**201** {id, application_id, items_rated}	401, 403, 409, 422
GET `/api/employers/{id}/trust-score`	Public	Employer trust score (published only from 3 responses)	–	**200** `TrustScore`	404

M8 Dashboards

Method and path	Who	What	Request	Success	Errors
GET `/api/candidate/dashboard`	Candidate	Candidate summary (applications by status, matches, pending feedback)	–	**200** `CandidateDashboard`	401, 403
GET `/api/employer/dashboard`	Employer	Employer summary (postings, applicants, trust score, verification)	–	**200** `EmployerDashboard`	401, 403
GET `/api/admin/dashboard`	Admin	Admin analytics (users, postings, verification queue, reports, charts)	–	**200** `AdminDashboard`	401, 403

M9 Notifications

Method and path	Who	What	Request	Success	Errors
GET `/api/notifications`	Candidate, Employer	Own notifications (unread count in meta)	–	**200** {data, meta}	401, 403
PATCH `/api/notifications/{id}/read`	Candidate, Employer	Mark one notification read	–	**204** no body	403, 404
PATCH `/api/notifications/read-all`	Candidate, Employer	Mark all notifications read	–	**204** no body	403

M10 Moderation

Method and path	Who	What	Request	Success	Errors
GET `/api/admin/users`	Admin	List users	–	**200** {data, meta}	403, 422
PATCH `/api/admin/users/{id}/suspend`	Admin	Suspend (revokes all tokens) or reactivate a user	{suspended*}	**200** `User`	403, 404, 422
POST `/api/job-postings/{id}/reports`	Candidate	Report a posting	{reason*}	**201** `Report`	401, 403, 404, 422
GET `/api/admin/reports`	Admin	List reports	–	**200** {data, meta}	403, 422
GET `/api/admin/reports/{id}`	Admin	Report details	–	**200** `Report`	403, 404
PATCH `/api/admin/reports/{id}`	Admin	Dismiss or act on a report	{status*, note}	**200** `Report`	403, 404, 422
GET `/api/admin/feedback`	Admin	Feedback responses for moderation (anonymous – no author)	–	**200** {data, meta}	403, 422
PATCH `/api/admin/feedback/{id}/hide`	Admin	Hide or unhide a feedback response (recomputes the trust score)	{hidden*}	**204** no body	403, 404, 422

M11 Master data

Method and path	Who	What	Request	Success	Errors
GET `/api/categories`	Public	Job categories	–	**200** {data}	–
GET `/api/skills`	Public	The shared skills list with other spellings (Decision 2026-10-21)	–	**200** {data}	–
GET `/api/accommodations`	Public	Accommodation types grouped (retired ones excluded)	–	**200** {data}	–
POST `/api/admin/categories`	Admin	Create category	{name*}	**201** `Category`	422

Method and path	Who	What	Request	Success	Errors
PUT `/api/admin/categories/{id}`	Admin	Update category	{name*}	**200** `Category`	–
DELETE `/api/admin/categories/{id}`	Admin	Delete category (only if unused)	–	**204** no body	422
GET `/api/admin/accommodations`	Admin	All accommodation types, including retired ones (admin list for restoring)	–	**200** {data}	403
POST `/api/admin/accommodations`	Admin	Create accommodation type	{name*, group*, description, is_retired}	**201** `Accommodation`	–
PUT `/api/admin/accommodations/{id}`	Admin	Update accommodation type	{name*, group*, description, is_retired}	**200** `Accommodation`	–
DELETE `/api/admin/accommodations/{id}`	Admin	Delete if unused, otherwise retire (is_retired = true)	–	**200** `Accommodation`	–

M12 Saved jobs

Method and path	Who	What	Request	Success	Errors
GET `/api/candidate/saved-jobs`	Candidate	Saved postings	–	**200** {data, meta}	401, 403
POST `/api/candidate/saved-jobs/{jobId}`	Candidate	Save a posting	–	**200** `JobPosting`	401, 403, 404
DELETE `/api/candidate/saved-jobs/{jobId}`	Candidate	Remove a saved posting	–	**204** no body	401, 403

M13 Accessibility settings

Method and path	Who	What	Request	Success	Errors
PUT `/api/me/display-prefs`	Signed in	Save the display settings with the account	`DisplayPrefs`	**200** `DisplayPrefs`	401, 422
PUT `/api/me/email-notifications`	Candidate, Employer	Turn the emails for notifications on or off	{enabled*}	**200** {enabled*}	401, 403, 422

Insights

Method and path	Who	What	Request	Success	Errors
GET `/api/insights/job-postings/{id}`	Public	Workplace Accessibility Snapshot (mashup of Core, PSGC, Nominatim, Overpass)	–	**200** `Snapshot`	404, 503
GET `/api/insights/locations/{level}`	Public	PSGC location lists for pickers	–	**200** `[LocationItem]`	404, 422, 503

Internal

Method and path	Who	What	Request	Success	Errors
GET `/internal/job-postings/{id}/summary`	Service key	Posting summary for the Insights Service (service key only; 404 if not open)	–	**200** `InternalPostingSummary`	401, 404
GET `/internal/places/{code}`	Service key	City or municipality with province/region codes and centre point, for Core (Decision 2026-10-21)	–	**200** {code, city, province, region, province_code, region_code, lat, lon, precision, +1}	401, 404, 503

5 Errors: envelope, codes and real examples

Every error from every service – and from the gateway when a service cannot answer – has the same shape, so the app shows one kind of message and support can trace it by `request_id`.

```
{
  "error": {
    "code": "ALREADY_APPLIED",
    "message": "You have already applied to this posting.",
    "request_id": "54045e98-7d4c-4f03-a21d-60841b55c7b6"
  }
}
```

422 validation errors keep Laravel's `message` and `errors` (messages per field, shown next to the form fields) and add the envelope's `code` and `request_id`.

HTTP	Code	When	Raised by
400	`BAD_REQUEST`	A request Core cannot process (a malformed JSON body is read as empty and gives 422 instead)	Core
401	`UNAUTHENTICATED`	No token, a revoked token, or a wrong service key on `/internal`	Core
403	`FORBIDDEN`	Signed in, but the role or ownership does not allow it	Core
403	`ACCOUNT_SUSPENDED`	The account was suspended by an admin (log-in refused, tokens revoked)	Core
404	`NOT_FOUND`	Unknown id, a posting that is not visible, or `/internal` through the gateway	Core, Insights, Gateway
409	`ALREADY_APPLIED`	The candidate applied to this posting before	Core
409	`FEEDBACK_EXISTS`	Feedback for this hire was already sent	Core
409	`HIRING_REPORT_PENDING`	A hiring report for this application is already waiting for an answer	Core
409	`HIRING_REPORT_CLOSED`	The hiring report was already answered or cancelled	Core
422	`VALIDATION_FAILED`	Input rules failed; `errors` lists the messages per field	Core, Insights
422	`INVALID_STATUS_TRANSITION`	The stage cannot move that way (stages only move forward, e.g. hired → reviewed), or `hired` is set directly – a hire needs a confirmed hiring report	Core
422	`PROFILE_INCOMPLETE`	Applying without a complete profile (place, at least 3 skills, education, work preferences)	Core
422	`RESUME_REQUIRED`	Applying without a résumé on file	Core
422	`POSTING_CLOSED`	Applying to a closed posting or after its closing date	Core
422	`FEEDBACK_NOT_OPEN`	Feedback before day 30 after the confirmed start date, or the application is not hired	Core
422	`REGISTRY_INVALID_REQUEST`	The SOAP registry refused the input (`Client.Validation` fault)	Core
429	`RATE_LIMITED`	Too many requests; wait for `Retry-After` seconds	Gateway, Core (log-in)
500	`SERVER_ERROR`	Unexpected error; generic message, never a stack trace	any
502	`REGISTRY_UNAVAILABLE`	The registry is down, timed out (5 s) or sent a `Server` fault	Core
502	`REGISTRY_AUTH_FAILED`	The registry refused Core's WS-Security credentials	Core

HTTP	Code	When	Raised by
502	`UPSTREAM_ERROR`	A service answered with something the gateway could not pass on	Gateway
503	`SERVICE_UNAVAILABLE`	The service behind the gateway cannot be reached (or Insights cannot reach Core)	Gateway, Insights
504	`UPSTREAM_TIMEOUT`	The service did not answer in time (Core 10 s, Insights 15 s)	Gateway

Real examples

401 – no token

Protected route without a token.

REQUEST

```
GET http://127.0.0.1:8080/api/me
```

RESPONSE

```
HTTP 401
content-type: application/json
x-request-id: c0b836e3-2c9c-46e5-8d66-6c2f11a142dd
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "UNAUTHENTICATED",
    "message": "Unauthenticated.",
    "request_id": "c0b836e3-2c9c-46e5-8d66-6c2f11a142dd"
  }
}
```

403 – wrong role

A candidate calls an admin endpoint.

REQUEST

```
GET http://127.0.0.1:8080/api/admin/users
Authorization: Bearer 6|•••••• (hidden)
```

RESPONSE

```
HTTP 403
content-type: application/json
x-request-id: 7102e2b0-cf21-406e-8bbd-399d38008d80
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "FORBIDDEN",
    "message": "This action is unauthorized.",
    "request_id": "7102e2b0-cf21-406e-8bbd-399d38008d80"
  }
}
```

404 – unknown posting

A posting id that does not exist.

REQUEST

```
GET http://127.0.0.1:8080/api/job-postings/999999
```

RESPONSE

```
HTTP 404
content-type: application/json
x-request-id: c29aace2-a925-4f0f-bb76-9b4178c818cc
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "NOT_FOUND",
    "message": "The requested resource was not found.",
    "request_id": "c29aace2-a925-4f0f-bb76-9b4178c818cc"
  }
}
```

409 – applied twice

The candidate applies to the same posting twice.

REQUEST

```
POST http://127.0.0.1:8080/api/job-postings/1/applications
Authorization: Bearer 6|•••••• (hidden)
Content-Type: application/json

{}
```

RESPONSE

```
HTTP 409
content-type: application/json
x-request-id: 6c98bf80-638d-441d-a8c8-bbe2df9682af
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "ALREADY_APPLIED",
    "message": "You have already applied to this posting.",
    "request_id": "6c98bf80-638d-441d-a8c8-bbe2df9682af"
  }
}
```

422 – validation

Registration with missing and invalid fields – Laravel's `errors` object is kept for the form, plus the envelope.

REQUEST

```
POST http://127.0.0.1:8080/api/register
Content-Type: application/json

{
  "role": "candidate",
  "email": "not-an-email",
  "password": "•••••• (hidden)"
}
```

RESPONSE

```
HTTP 422
content-type: application/json
x-request-id: 3e22a7a2-d2e2-4bb8-9501-0395a1180dce
ratelimit-policy: 300;w=60
vary: Origin

{
  "message": "The given data was invalid.",
  "error": {
    "code": "VALIDATION_FAILED",
    "request_id": "3e22a7a2-d2e2-4bb8-9501-0395a1180dce"
  },
  "errors": {
    "name": [
      "The name field is required."
    ],
    "email": [
      "The email field must be a valid email address."
    ],
    "password": [
      "The password field confirmation does not match.",
      "The password field must be at least 8 characters."
    ],
    "consent": [
      "You must agree to the data privacy notice to create an
account."
    ]
  }
}
```

422 – status change not allowed

The employer tries to move a hired application back to reviewed.

REQUEST

```
PATCH http://127.0.0.1:8080/api/applications/7/status
Authorization: Bearer 7|•••••• (hidden)
Content-Type: application/json

{
  "status": "reviewed"
}
```

RESPONSE

```
HTTP 422
content-type: application/json
x-request-id: e616637f-608d-46a6-8c3a-d97f1a0d1683
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "INVALID_STATUS_TRANSITION",
    "message": "An application that is hired cannot be change
d to reviewed.",
    "request_id": "e616637f-608d-46a6-8c3a-d97f1a0d1683"
  }
}
```

422 – hired cannot be set directly

The employer tries to set `hired` through the status endpoint – a hire is reported by one side and confirmed by the other.

REQUEST

```
PATCH http://127.0.0.1:8080/api/applications/8/status
Authorization: Bearer 7|•••••• (hidden)
Content-Type: application/json

{
  "status": "hired"
}
```

RESPONSE

```
HTTP 422
content-type: application/json
x-request-id: 607a4340-2979-4400-bff5-190a3bb549c9
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "INVALID_STATUS_TRANSITION",
    "message": "An application becomes hired only when both p
arties confirm a hiring report.",
    "request_id": "607a4340-2979-4400-bff5-190a3bb549c9"
  }
}
```

409 – a report is already waiting

The candidate reports the hire while the employer's report still waits for the candidate's answer.

REQUEST

```
POST http://127.0.0.1:8080/api/applications/8/hiring-reports
Authorization: Bearer 6|•••••• (hidden)
Content-Type: application/json

{
  "start_date": "2026-10-31"
}
```

RESPONSE

```
HTTP 409
content-type: application/json
x-request-id: 6a4da8e1-1efa-4873-83df-d2371a9b6d7b
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "HIRING_REPORT_PENDING",
    "message": "A hiring report is already waiting for an ans
wer.",
    "request_id": "6a4da8e1-1efa-4873-83df-d2371a9b6d7b"
  }
}
```

422 – feedback before day 30

The candidate rates the employer right after confirming the hire – the start date is a week ahead, so the form opens 37 days from now.

REQUEST

```
POST http://127.0.0.1:8080/api/applications/7/feedback
Authorization: Bearer 6|•••••• (hidden)
Content-Type: application/json

{
  "items": [
    {
      "accommodation_id": 1,
      "rating": "delivered"
    }
  ]
}
```

RESPONSE

```
HTTP 422
content-type: application/json
x-request-id: ee5d5e9b-c4a4-4942-b019-6375e64eb51a
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "FEEDBACK_NOT_OPEN",
    "message": "Feedback opens on 2026-11-30, 30 days after y
our confirmed start date.",
    "request_id": "ee5d5e9b-c4a4-4942-b019-6375e64eb51a"
  }
}
```

422 – wrong password

Wrong email or password – the same message for both cases, and a suspended account is reported only after a correct password.

REQUEST

```
POST http://127.0.0.1:8080/api/login
Content-Type: application/json

{
  "email": "nobody@opendoor.test",
  "password": "•••••• (hidden)"
}
```

RESPONSE

```
HTTP 422
content-type: application/json
x-request-id: 1718422e-7998-456c-a904-0ec2ec82ceb4
ratelimit-policy: 300;w=60
vary: Origin

{
  "message": "The given data was invalid.",
  "error": {
    "code": "VALIDATION_FAILED",
    "request_id": "1718422e-7998-456c-a904-0ec2ec82ceb4"
  },
  "errors": {
    "email": [
      "These credentials do not match our records."
    ]
  }
}
```

429 – too many log-in attempts

Six log-in attempts in a minute for one email: Core's log-in limit.

REQUEST

```
POST http://127.0.0.1:8080/api/login
Content-Type: application/json

{
  "email": "nobody@opendoor.test",
  "password": "•••••• (hidden)"
}
```

RESPONSE

```
HTTP 429
content-type: application/json
x-request-id: e40220ef-5120-4a0c-955e-375259f5b6d9
retry-after: 60
ratelimit-policy: 300;w=60
vary: Origin

{
  "error": {
    "code": "RATE_LIMITED",
    "message": "Too many requests. Please try again later.",
    "request_id": "e40220ef-5120-4a0c-955e-375259f5b6d9"
  }
}
```

Core down – any /api request

Core is not running: the gateway answers for it with the same envelope.

REQUEST

```
GET http://127.0.0.1:8099/api/job-postings
```

RESPONSE

```
HTTP 503
content-type: application/json; charset=utf-8
x-request-id: 6b9c64b2-30b7-4aa4-9e1c-76ccf8537a87
ratelimit-policy: 120;w=60
vary: Origin

{
  "error": {
    "code": "SERVICE_UNAVAILABLE",
    "message": "The service is temporarily unavailable. Pleas
e try again shortly.",
    "request_id": "6b9c64b2-30b7-4aa4-9e1c-76ccf8537a87"
  }
}
```

Health with Core and Insights down

The gateway keeps answering and reports which service is down.

REQUEST

```
GET http://127.0.0.1:8099/api/health
```

RESPONSE

```
HTTP 200
content-type: application/json; charset=utf-8
x-request-id: 8f135d78-4835-4d03-9e2e-bec082976500
ratelimit-policy: 120;w=60
vary: Origin

{
  "status": "degraded",
  "services": {
    "core": "down",
    "insights": "down"
  }
}
```

6 Legacy integration: the SOAP Business Registry

Service	Simulated DTI/SEC business registry (no public government API exists), PHP `SoapServer`, SQLite data. Only Core calls it.
Contract	WSDL at `/soap?wsdl`, SOAP 1.1, document/literal, namespace `urn:opendoor:registry:v1`.
Operation	`VerifyBusinessRegistration` (RegistrationNumber, RegistryType DTI|SEC, BusinessName) → Found, Status (ACTIVE · EXPIRED · CANCELLED · NOT_FOUND), RegisteredName, DateRegistered, NameMatchScore 0–1, CheckedAt.
Security	WS-Security `UsernameToken` header (user and password from `REGISTRY_SOAP_USER` / `REGISTRY_SOAP_PASSWORD`); the registry never logs the password. In production it is reachable only on Railway's private network.
Faults	`Client.Authentication` (credentials), `Client.Validation` (bad input), `Server` (internal, generic message).
Core side	`BusinessRegistryClient` (PHP `SoapClient`, 5 s timeout) adds the header, stores every answer in `registry_checks` (with the raw XML) and maps faults to API errors: authentication → 502 `REGISTRY_AUTH_FAILED`, validation → 422 `REGISTRY_INVALID_REQUEST`, server fault / timeout / connection error → 502 `REGISTRY_UNAVAILABLE` (stored as an `ERROR` check).
Decision	The result is advice. The admin approves or rejects; status ≠ ACTIVE or a name match below 0.80 is highlighted.

As the admin sees it (REST)

`PATCH /api/admin/employers/{id}/verify` with `{"action":"check"}` answers `{"registry": {"status", "registered_name", "name_match", "checked_at"}, "verified_at"}`; `approve` / `reject` (with `reason`) decide. The Run & Demo Guide, steps 6.7–6.8, shows the screens, logs and database rows.

The SOAP messages (Core ↔ registry)

Request and response – an active SEC registration

Core's request with the WS-Security header (user name and password hidden), then the registry's answer.

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ns1="urn:opendoor:registry:v1" xmlns:ns2
="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
  <SOAP-ENV:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
      <wsse:UsernameToken>
        <wsse:Username>•••••• (hidden)</wsse:Username>
        <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#PasswordTex
t">•••••• (hidden)</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </SOAP-ENV:Header>
  <SOAP-ENV:Body>
    <ns1:VerifyBusinessRegistrationRequest>
      <ns1:RegistrationNumber>CS201912345</ns1:RegistrationNumber>
      <ns1:RegistryType>SEC</ns1:RegistryType>
      <ns1:BusinessName>Kalinga Foods Inc.</ns1:BusinessName>
    </ns1:VerifyBusinessRegistrationRequest>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ns1="urn:opendoor:registry:v1">
  <SOAP-ENV:Body>
    <ns1:VerifyBusinessRegistrationResponse>
      <ns1:Found>true</ns1:Found>
      <ns1:Status>ACTIVE</ns1:Status>
      <ns1:RegisteredName>KALINGA FOODS, INC.</ns1:RegisteredName>
      <ns1:DateRegistered>2019-03-14</ns1:DateRegistered>
      <ns1:NameMatchScore>1</ns1:NameMatchScore>
      <ns1:CheckedAt>2026-10-24T00:28:35+08:00</ns1:CheckedAt>
    </ns1:VerifyBusinessRegistrationResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Response – an expired DTI registration

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ns1="urn:opendoor:registry:v1">
  <SOAP-ENV:Body>
    <ns1:VerifyBusinessRegistrationResponse>
      <ns1:Found>true</ns1:Found>
      <ns1:Status>EXPIRED</ns1:Status>
      <ns1:RegisteredName>SARI-SARI DIGITAL SERVICES</ns1:RegisteredName>
      <ns1:DateRegistered>2015-02-10</ns1:DateRegistered>
      <ns1:NameMatchScore>1</ns1:NameMatchScore>
      <ns1:CheckedAt>2026-10-24T00:28:35+08:00</ns1:CheckedAt>
    </ns1:VerifyBusinessRegistrationResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Response – an unknown number

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ns1="urn:opendoor:registry:v1">
  <SOAP-ENV:Body>
    <ns1:VerifyBusinessRegistrationResponse>
      <ns1:Found>false</ns1:Found>
      <ns1:Status>NOT_FOUND</ns1:Status>
      <ns1:NameMatchScore>0</ns1:NameMatchScore>
      <ns1:CheckedAt>2026-10-24T00:28:35+08:00</ns1:CheckedAt>
    </ns1:VerifyBusinessRegistrationResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Fault – invalid input

Core turns this into 422 `REGISTRY_INVALID_REQUEST` with the registry's reason.

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <SOAP-ENV:Fault>
      <faultcode>SOAP-ENV:Client.Validation</faultcode>
      <faultstring>RegistrationNumber is required and must be 5-20 letters or digits</faultstring>
    </SOAP-ENV:Fault>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Fault – wrong credentials

Core turns this into 502 `REGISTRY_AUTH_FAILED` and logs a hint to check `REGISTRY_SOAP_USER` / `REGISTRY_SOAP_PASSWORD`.

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
    <SOAP-ENV:Fault>
      <faultcode>SOAP-ENV:Client.Authentication</faultcode>
      <faultstring>Invalid WS-Security credentials</faultstring>
    </SOAP-ENV:Fault>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

7 The mashup: Workplace Accessibility Snapshot

`GET /api/insights/job-postings/{id}` (public) combines four sources into one answer: what the employer promises, what hired employees report, and what is really around the workplace.

Source	Gives	Call	Limits	Cache	When it fails
Core (internal)	The posting: address, PSGC code, accommodations, trust score	`GET /internal/job-postings/{id}/summary` + `X-Service-Key`	3 s	never cached	404 → snapshot 404; anything else → 503 (the only required source)
PSGC API	Official place names (region, province, city)	`psgc.gitlab.io/api`	cooldown 60 s after a failure	7 days	expired copy (`stale`) → bundled lists → Core's label
Nominatim (OpenStreetMap)	Coordinates of the workplace address	`/search?q=…&countrycodes=ph`, User-Agent with contact email	1 request per second (queue), 5 s	30 days	city centre (`precision: city`) or `unavailable`
Overpass (OpenStreetMap)	Wheelchair-accessible places within 500 m	Overpass QL `around:500`, 2 servers (second after 5 s or a failure)	25 s per server, 60 s cooldown	24 hours	expired copy (`stale`), else `unavailable`; background retries after 90 s, 5 min, 15 min

- **Order:** Core first (required) → PSGC → Nominatim → Overpass. The whole snapshot has a **12 s budget** (the gateway allows 15 s): a source still running at the deadline counts as `unavailable`, but its request goes on and fills the cache for the next visitor.
- **Cache:** Insights' own MySQL database `insights_cache` (tables `geocode_cache`, `overpass_cache`, `psgc_cache`). Two visitors asking for the same place at once share one request. Posting data from Core is never cached.
- **Output:** `location`, `employer_claims` (accommodations by group), `community_verification` (trust score, null until published), `nearby_accessible` (nearest 3 per category within 500 m: transit stops, toilets, parking, other), `sources`, `disclaimer`, `generated_at`.

The `sources` flags

Value	Meaning
`ok`	Fetched now from the source
`cached`	Served from the cache, still fresh
`stale`	The source failed; an expired cached copy is shown
`unavailable`	The source failed or ran out of the 12 s budget and nothing is cached; that part of the snapshot is empty

A real snapshot

REQUEST

```
GET http://127.0.0.1:8080/api/insights/job-postings/2
```

RESPONSE

```
HTTP 200
content-type: application/json; charset=utf-8
x-request-id: 5c35627a-6f41-40c2-9ffa-9bc47f0e5d7e
ratelimit-policy: 300;w=60
vary: Origin

{
  "job_posting_id": 2,
  "location": {
    "psgc_code": "1381300000",
    "label": "Quezon City, National Capital Region (NCR)",
    "lat": 14.636788,
    "lon": 121.031402,
    "precision": "street"
  },
  "employer_claims": {
    "work_arrangement": [
      "Flexible hours",
```

```
      "Hybrid work"
    ],
    "communication": [
      "Sign language interpreter",
      "Written or captioned meetings"
    ]
  },
  "community_verification": {
    "trust_score": 79,
    "based_on": 3,
    "breakdown": [
      {
        "accommodation_id": 2,
        "name": "Accessible restroom",
        "delivered": 2,
        "partial": 0,
        "not_delivered": 0,
        "of": 2
      },
      {
        "accommodation_id": 10,
        "name": "Flexible hours",
        "delivered": 0,
        "partial": 1,
        "not_delivered": 0,
        "of": 1
      },
      "… 5 more"
    ]
  },
  "nearby_accessible": null,
  "sources": {
    "core": "ok",
    "psgc": "cached",
    "nominatim": "cached",
    "overpass": "unavailable"
  },
  "disclaimer": "Map data from OpenStreetMap contributors (ODbL); coverage may be incomplete.",
  "generated_at": "2026-10-24T00:28:33+08:00"
}
```


Insights → Core: the internal summary

The service-to-service call behind the snapshot. Called directly on Core (the gateway never routes `/internal`; without the key Core answers 401).

REQUEST

```
GET http://127.0.0.1:8001/internal/job-postings/2/summary
X-Service-Key: •••••• (hidden)
```

RESPONSE


```
HTTP 200
content-type: application/json
x-request-id: e9c78771-cce9-484a-9b6a-26d20721eb76

{
  "id": 2,
  "address": "25 Scout Tobias St., South Triangle",
  "psgc_code": "1381300000",
  "location_label": "Quezon City, National Capital Region (NCR)",
  "accommodations": [
    {
      "id": 10,
      "name": "Flexible hours",
      "group": "work_arrangement"
    },
    {
      "id": 9,
      "name": "Hybrid work",
      "group": "work_arrangement"
    },
    "… 2 more"
  ],
  "trust_score": {
    "published": true,
    "score": 79,
    "based_on": 3,
```

```
    "breakdown": [
      {
        "accommodation_id": 2,
        "name": "Accessible restroom",
        "delivered": 2,
        "partial": 0,
        "not_delivered": 0,
        "of": 2
      },
      {
        "accommodation_id": 10,
        "name": "Flexible hours",
        "delivered": 0,
        "partial": 1,
        "not_delivered": 0,
        "of": 1
      },
      "… 5 more"
… (3 more lines)
```


Location lists (PSGC)

`GET /api/insights/locations/{regions|provinces|cities}?parent=<10-digit code>` feeds the location pickers. `X-Data-Source` says where the list came from: `cached`, `ok`, `stale` or `fallback` (lists bundled with the service).

REQUEST

```
GET http://127.0.0.1:8080/api/insights/locations/regions
```

RESPONSE


```
HTTP 200
content-type: application/json; charset=utf-8
x-request-id: 725d046d-6d5b-4888-9f40-c9c9b201a4aa
ratelimit-policy: 300;w=60
x-data-source: cached
vary: Origin

[
  {
    "code": "1900000000",
    "name": "Bangsamoro Autonomous Region in Muslim Mindanao
(BARMM)",
    "level": "region"
  },
  {
    "code": "0500000000",
    "name": "Bicol Region (Region V)",
    "level": "region"
  },
  "… 15 more"
]
```

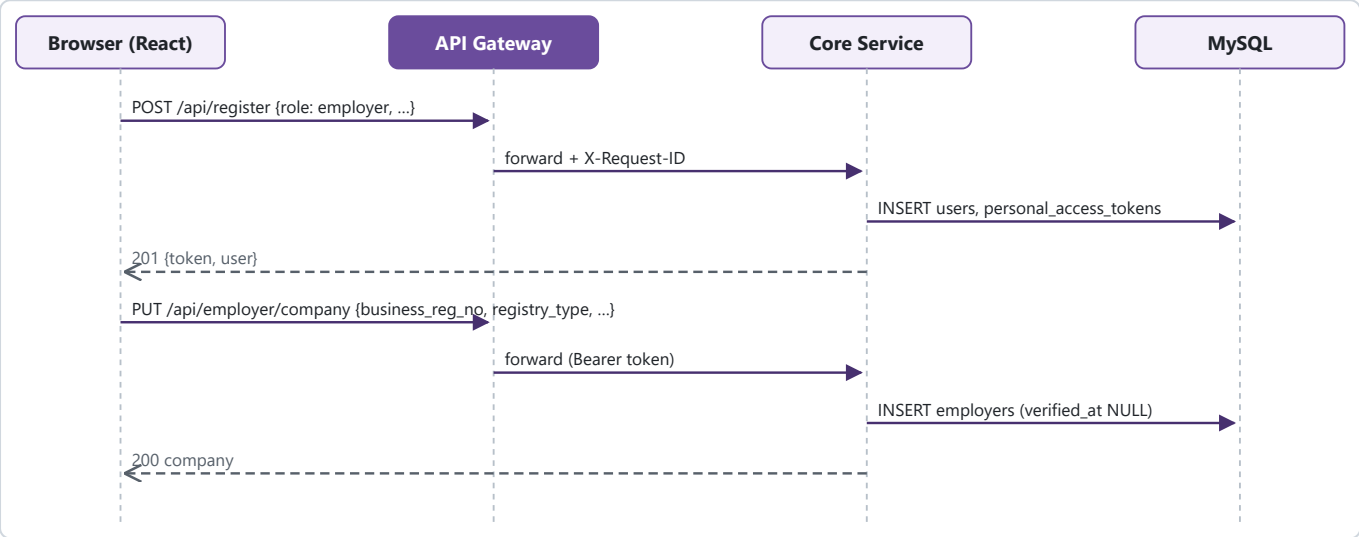
8 End-to-end scenario through every component

One employer and one candidate from registration to a published trust score. Each phase shows the calls in order; solid arrows are requests, dashed arrows answers.

Participants: Browser (React) · API Gateway · Core Service · MySQL · SOAP Registry · Insights · External APIs (External APIs = PSGC, Nominatim, Overpass).

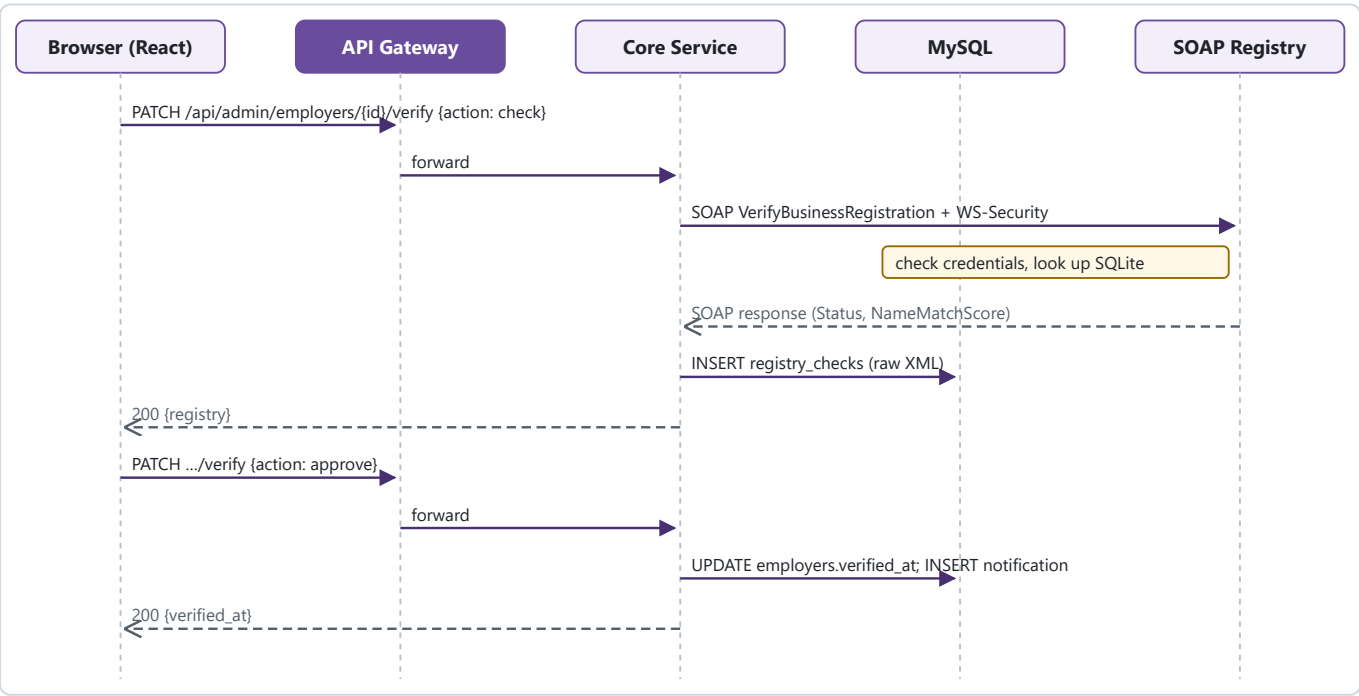
8.1 Employer registers and fills in the company profile

The employer creates an account (consent required) and enters the business registration number. The company starts unverified.



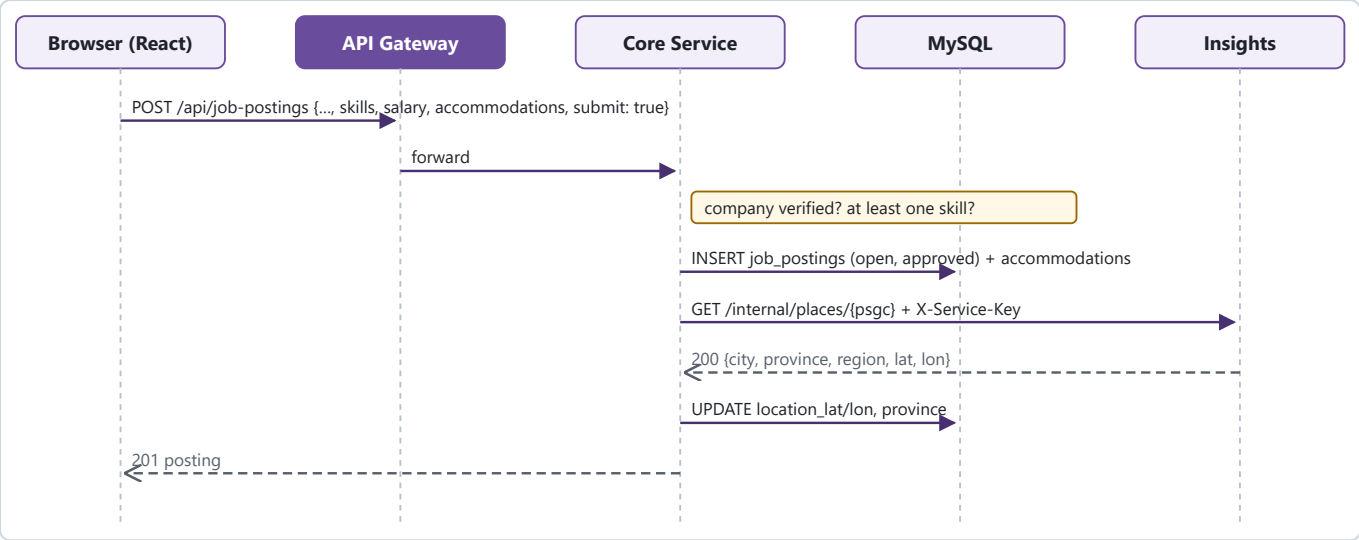
8.2 Admin checks the employer with the SOAP registry and approves

Core is the SOAP client. The registry answers from its own data; Core stores the answer and the admin decides.



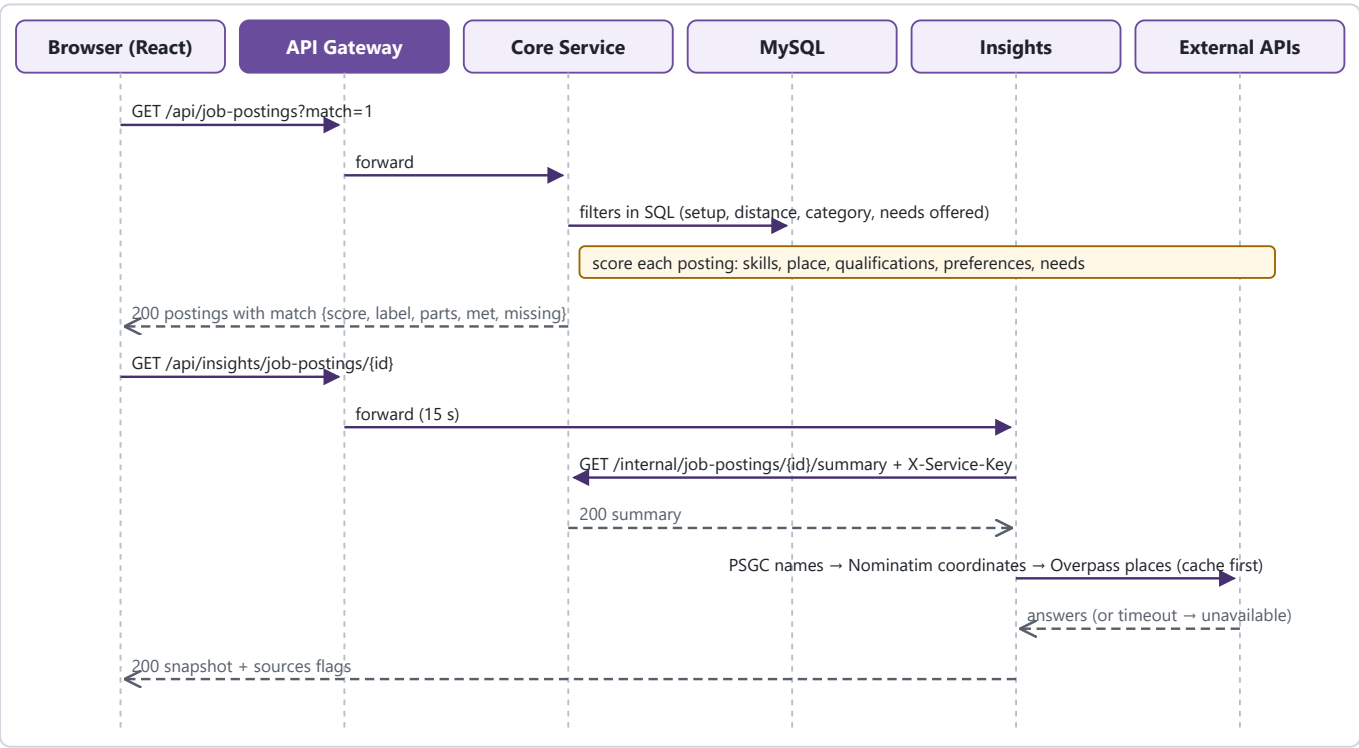
8.3 The verified employer publishes a job

There is no approval step: only a verified company can publish, and its posting is visible at once. Core asks Insights for the city-centre point of the place (service key) so that distance searches work; if Insights does not answer in 4 s the posting is still saved and a scheduled job fills the point later.



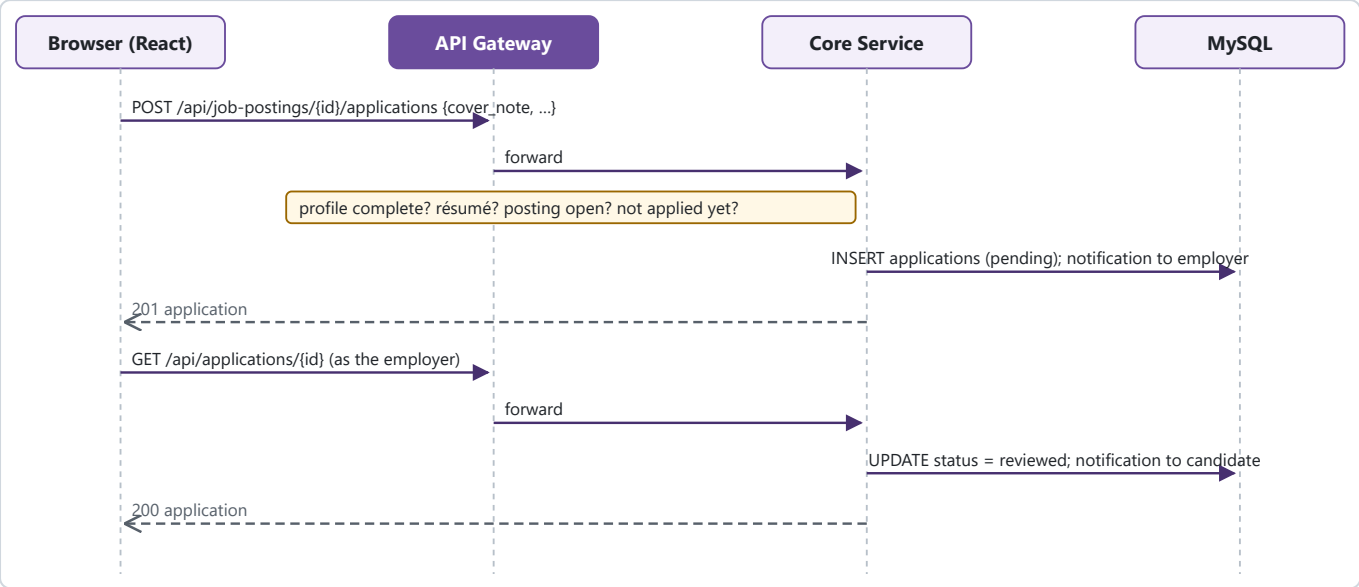
8.4 Candidate finds the job by its match score and opens the snapshot

Matching runs in Core on the candidate's profile and needs: skills 40, location 20, qualifications 15, work preferences 10, accommodation needs 15 – a score from 0 to 100. The snapshot is the mashup: Insights combines Core's posting with three public APIs.



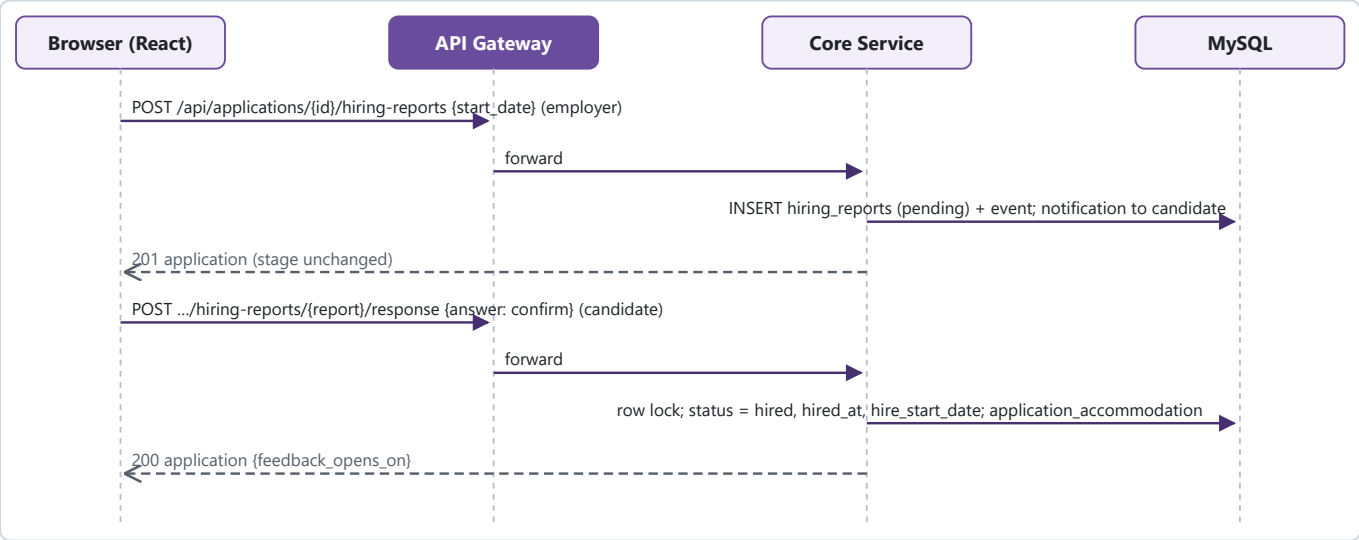
8.5 Candidate applies; employer reviews

Applying needs a complete profile and a résumé. Every stage change is checked against the workflow (stages only move forward) and notifies the other side.



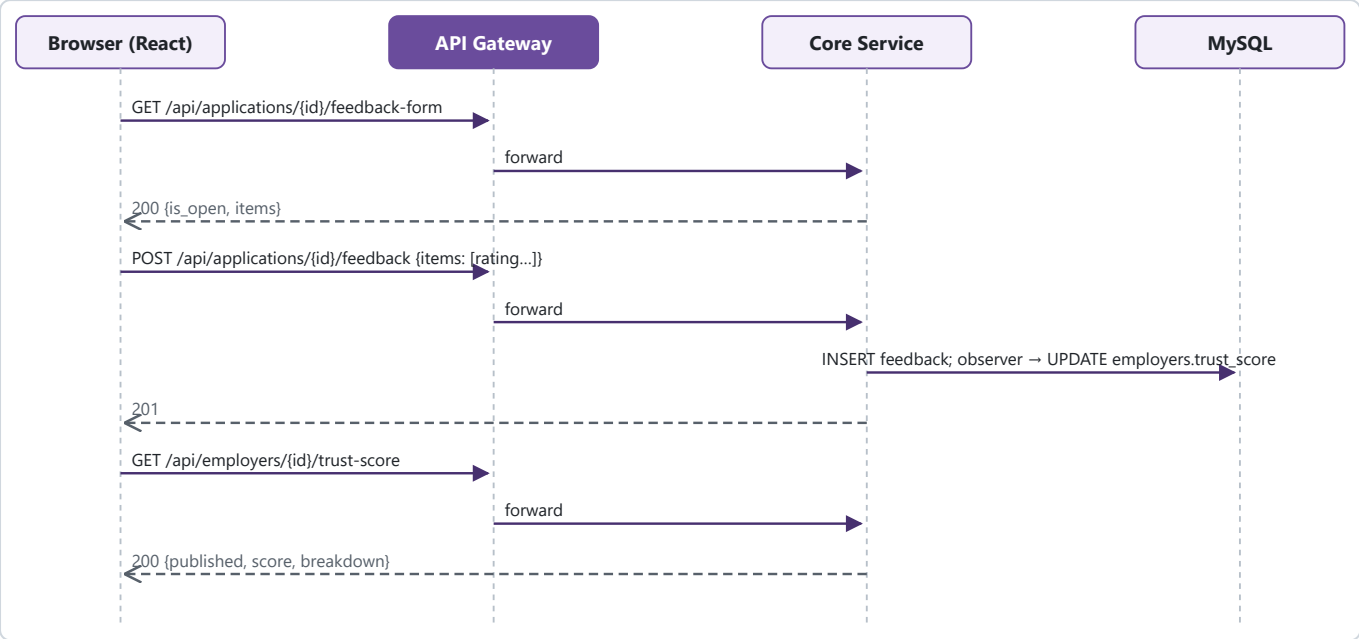
8.6 The hire is reported by one side and confirmed by the other

A hire is never set directly. The employer (or the candidate) reports it with the start date; the other side answers Confirm, Not yet (optionally a new date) or Incorrect. Only Confirm makes the application Hired – the rating opens on day 30 after that start date.



8.7 From day 30 after the start date: feedback builds the trust score

The hired candidate rates each promised accommodation anonymously. An observer recomputes the employer's score; it is published from 3 responses and then appears on the public page and in the snapshot.



9 Security summary

Exposure	Only the gateway is public. Core, Insights, the registry and MySQL are on Railway's private network. `/internal/*` is never routed (404).
Authentication	Laravel Sanctum bearer tokens, stored as SHA-256 hashes. Passwords hashed with bcrypt. Log-out deletes the token; a password reset signs the user out everywhere; suspension revokes all tokens. Tokens have no time limit otherwise (Sanctum `expiration` is not set).
Authorization	On the server, in three layers: `auth:sanctum` → role middleware → Laravel Policies (ownership, e.g. an employer sees only applications to their own postings). The React route guards are only navigation.
Service-to-service	Insights → Core (posting summary) and Core → Insights (city-centre points) with `X-Service-Key` (compared in constant time); Core → registry with a WS-Security UsernameToken. No service reads another service's database.
Input	Validation on every write (Laravel Form Requests): types, lengths, enums, existing ids. Uploads: résumé PDF ≤ 5 MB, logo PNG/JPG/WebP ≤ 2 MB, checked by extension and content type. Search text treats `%` and `_` literally; Eloquent uses bound parameters (no SQL injection).
Abuse	Gateway rate limit per IP; 5 log-in and 5 password-reset attempts per minute per email + IP. Forgot-password always answers the same and only queues a job, so neither the answer nor its timing reveals whether an account exists.
Browser	CORS only for the app's origin; the gateway removes `X-Powered-By` and the services' own CORS headers. HTTPS everywhere in production (Vercel, Railway).
Errors and logs	One error envelope with a request id, never a stack trace (`APP_DEBUG=false` in production). The gateway logs method, path, status and time – never headers or bodies, so tokens, needs and feedback are not logged.
Privacy (RA 10173)	Consent at registration. Matching uses accommodation needs, never the disability type (optional, hidden by default). Employers see a candidate's needs only through an application to their own posting. Feedback is anonymous and the score stays hidden until 3 responses. Emails carry no needs or disability details. Résumé text is read only to suggest skills and never stored.
Secrets	Only in `.env` files (git-ignored) and Railway/Vercel variables. CI generates its own per run. A gitleaks scan of the whole history found no leaks.

10 Questions we expect, with answers

Q1 **Why do you need an API gateway?**

It is the single public entry: one URL and one CORS rule for the app, rate limiting, request IDs for tracing, timeouts, and the same error format even when a service is down. It also hides the services – in production only the gateway has a public address. It routes by path: `/api/insights/*` to Insights, everything else under `/api` to Core.

Q2 **How does a request get from the screen to the database?**

React calls the gateway (Axios, bearer token). The gateway adds `X-Request-ID`, checks CORS and the rate limit and proxies to Core. Core authenticates the token (Sanctum), checks the role and the Policy, validates the input, runs the business rule and writes with Eloquent to MySQL, then answers JSON. Section 6 of the Run & Demo Guide shows this with real data.

Q3 **How do the services communicate with each other?**

REST/JSON between Insights and Core over the private `/internal` route with a service key; SOAP/XML between Core and the legacy registry with WS-Security; REST/JSON from Insights to the public PSGC, Nominatim and Overpass APIs. Services never share a database.

Q4 **Why SOAP for the registry?**

The project needs a legacy component, and government business registries are the typical case. There is no public DTI/SEC API, so we built a simulated registry that behaves like one: published WSDL, document/literal SOAP 1.1, WS-Security, SOAP faults. Core integrates it exactly like a real external SOAP service.

Q5 **What happens if the registry is down?**

Core waits at most 5 s, stores an `ERROR` check and answers 502 `REGISTRY_UNAVAILABLE`. The admin sees the message and can still decide by hand. Nothing else in the system depends on the registry.

Q6 **What is your mashup and why is it useful?**

The Workplace Accessibility Snapshot combines four sources into something none of them has alone: the employer's promised accommodations and the trust score from hired employees (Core), the official place name (PSGC), the coordinates (Nominatim) and wheelchair-accessible transit, toilets and parking nearby (Overpass/OpenStreetMap).

Q7 **What if an external API fails or is slow?**

The snapshot still answers 200. Each source is cached (30 days, 24 h, 7 days), a failed source gets a 60 s cooldown, Overpass has a second server, and the whole snapshot has a 12 s budget. What is missing is marked in `sources` (`stale` or `unavailable`) and the page says so. Failed places are retried in the background. Only Core is required.

Q8 **How do you make sure an employer cannot see other employers' applicants?**

Laravel Policies on the server. The application query is limited to the employer's own postings and `GET /applications/{id}` for anyone else returns 403. The Postman folder "Negative & Security Cases" (T09 plus role, ownership and privacy checks) tests this on every CI run.

Q9 **Do tokens expire?**

Not by time – Sanctum's expiration is not set, which is common for SPA tokens. A token ends when the user logs out, resets the password (all tokens) or is suspended (all tokens). They are stored only as hashes. A time limit would be a one-line config change.

Q10 **How do you protect personal data (RA 10173)?**

Consent at registration; matching by accommodation needs, never by disability type; the disability type is optional and hidden by default; employers see needs only through an application to their own posting; anonymous feedback with the score hidden until 3 responses; no bodies or tokens in logs; no needs in emails.

Q11 How does the match score work?

Core's `MatchingService` gives each posting 0–100 points for the signed-in candidate: skills 40 (the posting's required skills the candidate has, compared through the shared skills list and its aliases), location 20 (remote or same city full, same province 60 %, same region 30 %), qualifications 15 (minimum education and years of experience), work preferences 10 (setup and employment type) and accommodation needs 15 (share of her needs the posting offers). A part without data on either side is left out and the rest re-weighted. Labels: Strong ≥ 80, Good ≥ 60, Fair ≥ 40, else Low. The disability type is never used.

Q12 How do you know a hire really happened?

A hire needs both sides: one reports it with the start date (`POST /applications/{id}/hiring-reports`), the other confirms it. The employer cannot set `hired` directly (422), cannot answer its own report (403), and every report and answer is kept as history. Only a confirmed hire can rate the employer, from day 30 after the start date – so an employer cannot collect ratings from people it never hired.

Q13 How is the trust score calculated?

Each rated accommodation counts delivered = 1, partial = 0.5, not delivered = 0. The score is the average × 100, rounded to a whole percent, over all responses that an admin has not hidden. An observer recomputes it whenever feedback is saved or hidden; it is published from 3 responses.

Q14 How do you trace an error?

Every response carries `X-Request-ID`, also inside every error body. The gateway logs one line per request with that id, and Core and Insights receive it too, so one id finds the request in every log.

Q15 How did you test the integration?

Unit tests per service (PHPUnit – also on MySQL 8 –, Jest, Vitest, the registry runner: 907 tests on October 23); the Postman collection (SIA test plan T01–T16, 217 requests) through the gateway with Newman – in CI with PSGC, Nominatim and Overpass replaced by mock servers, and locally against the live APIs – plus a run with Overpass down (T13); JMeter load and stress tests on production; axe-core for accessibility (33 pages, 0 violations); and a validation script for the running stack.

Q16 Can someone bypass the gateway and call Core directly?

Not in production – Core has no public address. Locally it runs on its own port for development. Even then, every protected route needs a valid token and `/internal` needs the service key.

Q17 Why REST/JSON for the main API?

The React client works with JSON natively, REST maps cleanly to the resources (postings, applications, feedback), it is stateless (scales behind the gateway) and it is documented in OpenAPI. SOAP is used only where the legacy system requires it.

Q18 How is it deployed?

React on Vercel; gateway, Core, Insights, the registry and MySQL as five Railway services built from Dockerfiles, connected over Railway's private network. GitHub Actions runs the tests on every push.